UMBC CODEBOT '25 Workshop
Columbia, MD / 25-26 February 2025

Growing System Engineers in the Age of ChatGPT

William Bardon
UMBC
B277@umbc.edu

Abstract

Coding is one of many skills required in a system engineer, but it is fundamental. Artificial Intelligence (AI), specifically Large Language Models (LLMs) like ChatGPT, already demonstrate rudimentary coding ability. Emphasizing Test-driven Development practices with students restores a focus on problem analysis and verifies understanding.

Introduction

Large Language Models (LLMs) are advancing and can already generate usable programs in some situations [France24]. The Visual Studio Code Integrated Development Environment (IDE) sports an icon to activate Microsoft's AI Copilot. Despite a study as recent at Spring of 2024[Sandström] finding that Large Language Models (LLMs) were at best only capable of answering 40% of the questions in the 2023 Advent of Code (AOC) programming challenge, participants in AOC 2024 complained on Reddit about players high on the leaderboard using LLMs [RedditConsensus, RedditBan]. Considering the easy availability of LLM generated code, what, if anything can be done to ensure that students learn good development practices?

Building System Engineers

Programming is a first step towards becoming a system engineer. New programmers need to learn the syntax of a language in order to express solutions to problems in executable form. Computer science students study data structures to understand effective organization of data to support programs and algorithms to recognize opportunities to apply efficient approaches to specific problems. While specific problem details may change, programing exercises designed to reinforce particular algorithm and data structure implementations are readily applicable to solutions from LLMs trained on an abundance of similar examples.

Bardon, W. (2025). *Growing System Engineers in the Age of ChatGPT*. Paper presented at UMBC CodeBot '25, Columbia, MD. USA. Licensed under CC BY-NC-SA 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/

The temptation to turn to AI solvers conflicts with the pedagogical goal of reinforcing lessons with experience. A novice might rely on AI code generators to the point of not being able to detect flaws in AI results or to fix those flaws if identified. The mechanics of language syntax, data structures, and algorithms provide a baseline that encapsulates years of programming experience and build skills in assessing and correcting code. Students who bypass the foundation can be expected to struggle when confronted with more complicated and realistic projects.

Test-Driven Development

Turing award recipient C.A.R. Hoare [Hoare09] wrote,

> *"One thing I got spectacularly wrong. I could see that programs were getting larger, and I thought that testing would be an increasingly ineffective way of removing errors from them. I did not realize that the success of tests is that they test the programmer, not the program. Rigorous testing regimes rapidly persuade error-prone programmers (like me) to remove themselves from the profession. Failure in test immediately punishes any lapse in programming concentration, and (just as important) the failure count enables implementers to resist management pressure for premature delivery of unreliable code. The experience, judgment, and intuition of programmers who have survived the rigors of testing are what make programs of the present day useful, efficient, and (nearly) correct. Formal methods for achieving correctness must support the intuitive judgment of programmers, not replace it."*

Delivering solution code alone does not demonstrate proficiency. The student should be able to articulate how the solution code solves the problem and that it solves the problem correctly. Tests are implementations of the logical assertion of the program's correctness. Test quality demonstrates a student's understanding of the problem and the solution implementation. Creating good tests is a better proof of expertise than creating running code. To build successful tests, programmers must define:

- what the code being tested does
- acceptable and unacceptable inputs [Bratus17, Bratus16]
- rules of transformation from input to output
- invariant conditions for output
- representative test data
    - include corresponding pairs of inputs and outputs
    - confirm correct handling of invalid input
    - deliberately trigger special cases (such as null pointers and division by zero)

The resulting tests accumulate to run against each iteration of feature development, verifying that new code doesn't break existing code. Developers should be able to interpret requirements, specify them clearly, and define rigorously what working means, then prove logically that code meets requirements.

Growing System Engineers in the Age of ChatGPT © 2025 by William Bardon is licensed under CC BY-NC-SA 4.0  . To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/

Test-driven development (TDD) is a specific agile development methodology. One principle of TDD is writing the test before implementing the feature [Beck23]. Writing the test first encourages developers to write testable code. Writing the test first also demonstrates that the existing code does not pass the test and that passing the test is a result of feature implementation. Planning the tests first also supports logical structure in the assertions of correctness in the tests. The thought process of creating tests should lead the student to deeper understanding of the problem even if the implementation is then delegated to AI.

Scaling Up

Students' development scales as they advance in experience. Early assignments focus on single functions or small programs, but realism increases as they advance. Application complexity grows through successive layers. Individual functions take defined inputs and return output or cause side effects. Related functions and data structures can be combined in language specific constructs such as classes, packages, and modules. Applications may operate asynchronously through multi-threading or multiple processes. Cloud architectures (virtual machines or containers) or application stacks run on disparate physical or virtual computing platforms.

Testing and verification scale similarly:

- unit testing of individual functions
- integration testing of interacting components
- end-to-end testing of an entire system

Integration and end-to-end testing rely on tools that build and deploy systems in controlled environments where specified output from inputs can be captured and verified.

Writing application code is a small part of developing modern systems that operate in a cloud and consist of communities of communicating processes. Besides source code in a programming language, the developer must inform supporting tools for building, testing, deploying, and configuring system components, each in their own syntax. System supporting tools include build systems, remote execution and orchestration, executable packaging and versioning, and virtual infrastructure as code. Novices may invoke compilers from the command line or from an Integrated Development Environment (IDE), but, as Jeff Atwood pointed out, "F5 is not a build process"[Atwood07]. Each step in the delivery of a professional system should be repeatable and available for later review, hence automated with the automation code committed to a source control system and tagged by milestone or release build.

Final Thoughts

In Plato's *Phaedrus*, Socrates asserted that writing would "create forgetfulness" in students who would end up "having the show of wisdom without the reality"[Plato].

Growing System Engineers in the Age of ChatGPT © 2025 by William Bardon is licensed under CC BY-NC-SA 4.0 . To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/

> *"SOCRATES: ... And in this instance, you who are the father of letters, from a paternal love of your own children have been led to attribute to them a quality which they cannot have; for this discovery of yours will create forgetfulness in the learners' souls, because they will not use their memories; they will trust to the external written characters and not remember of themselves. The specific which you have discovered is an aid not to memory, but to reminiscence, and you give your disciples not truth, but only the semblance of truth; they will be hearers of many things and will have learned nothing; they will appear to be omniscient and will generally know nothing; they will be tiresome company, having the show of wisdom without the reality."*

LLMs offer a similar prospect for programmers. Perhaps the lesson we should learn from Plato is not to lament the loss of expertise in generating code, but to raise our perspective and appreciate tools that liberate developers from mundane concerns about syntax, allowing them to focus on larger scale design and problem-solving. LLMs are a continuation of trends such as IDES that suggest code fixes or linters that enforce good coding practices. Programmers are becoming system developers, leaving behind code syntax just as compilers liberated general purpose programmers from assembly language. The relevant skills become analysis and verification to recognize whether generated code actually solves the problem.

Will C/C++ become as anachronistic as COBOL? New languages such as Go [Golang] and Rust [Rust] promise memory safety without significant performance penalties. FBI and CISA specifically discourage using C and C++ because of memory risks [CISA24]. The "Rust for Linux" project works to translate the Linux Kernel into Rust [Rust-for-Linux]. In particular, "Rust for Linux" has generated friction with C Kernel developers because Rust's explicit object ownership semantics require greater specificity in the function interface than C [Speed24]. Developers of compilers and programming languages will continue to wrestle with the details of memory management, but compilers, garbage collectors, and language facilities relieve general application developers of that concern.

Conclusion

The critical skill at all levels boils down to analysis. Engineers assess a problem set and identify potential solutions. They analyze each potential solution to identify resource requirements, implementation steps, and validation criteria. They compare different solutions to identify the most cost-effective solution, while requiring viable solutions and reliable, secure operation.

Principles are similar at all levels of experience. Students can start with TDD at the function and small application level. Instruction focus shifts to the analytical process, including the development of test input representing both positive and negative normal operation, as well as boundaries and special cases.

Growing System Engineers in the Age of ChatGPT © 2025 by William Bardon is licensed under CC BY-NC-SA 4.0 . To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/

References

[Atwood07] Jeff Atwood, “The F5 Key is not a Build Process.” 31 OCT 2007. Published on Coding Horror. Retrieved on 1 DEC 2025 from https://blog.codinghorror.com/the-f5-key-is-not-a-build-process/

[Beck23] Kent Beck, “Canon TDD.” 11 DEC 23. on Software Design: Tidy First? Retrieved on 31 DEC 24 from https://tidyfirst.substack.com/p/canon-tdd

[Bratus16] Sergey Bratus, Adam J. Crain, Sven M. Hallberg, Daniel P. Hirsch, Meredith L. Patterson, Maxwell Koo, Sean W. Smith, “Implementing a vertically hardened DNP3 control stack for power applications.” In the Second Annual Industrial Control System Security (ICSS) Workshop at ACSAC 2016, Dec. 2016, Los Angeles. Retrieved on 4 NOV 24 from http://langsec.org/dnp3/dnp3-icss2016.pdf

[Bratus17] Sergey Bratus, Lars Hermerschmidt, Sven M. Hallberg, Michael E. Locasto, Falcon D. Momot, Meredith L. Patterson, and Anna Shubina, “Curing the Vulnerable Parser: Design Patterns for Secure Input Handling.” 2017. Retrieved on 4 NOV 24 from https://langsec.org/papers/curing-the-vulnerable-parser.pdf

[CISA24] Cybersecurity and Infrastructure Security Agency & Federal Bureau of Investigation, Product Security Bad Practices. October 2024. Retrieved on 4 NOV 24 from https://www.cisa.gov/sites/default/files/2024-10/joint-guidance-product-security-bad-practices-508c.pdf

[France24] Stephen L. France, “Navigating software development in the ChatGPT and GitHub Copilot era.” Business Horizons, Volume 67, Issue 5, 2024, pp 649-661, Retrieved on 22 DEC 2024 from https://doi.org/10.1016/j.bushor.2024.05.009

[Golang] golang.org, Documentation. The Go Programming Language. Retrieved on 2 JAN 2025 from https://go.dev/doc/

[Hoare09] Hoare, C.A.R., "Viewpoint Retrospective: an axiomatic basis for computer programming", October 2009, Communications of the Association for Computing Machinery, 52:10, pp30–32. https://doi.org/10.1145/1562764.1562779 Retrieved on 27 DEC 2024 from https://cacm.acm.org/author/c-a-r-hoare/

[Jin24] Kailun Jin, Chung-Yu Wang, Hung Viet Pham, Hadi Hemmati, “Can ChatGPT Support Developers? An Empirical Evaluation of Large Language Models for Code Generation”. York University, Toronto, ON Canada. in 21st International Conference on Mining Software Repositories (MSR ’24), April 15–16, 2024, Lisbon, Portugal. pp167-171. doi: 10.1145/3643991.3645074; Retrieved on 22 DEC 24 from https://arxiv.org/html/2402.11702v2

[Plato] Plato, Phaedrus. Translated by Benjamin Jowett. Project Gutenberg, 2008. Retrieved on 1 JAN 2025 from https://www.gutenberg.org/files/1636/1636.txt

Growing System Engineers in the Age of ChatGPT © 2025 by William Bardon is licensed under CC BY-NC-SA 4.0 . To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/

[RedditBan] Reddit discussion, “Could we ban the known LLM users from the leaderboard?” Retrieved on 22 DEC 2024 from https://www.reddit.com/r/adventofcode/comments/1h9195h/could_we_ban_the_known_llm_users_from_the

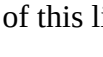
[RedditConsensus] Reddit discussion, “What is the current consensus of AI submitted code appearing on leaderboards?” Retrieved on 22 Dec 2024 from https://www.reddit.com/r/adventofcode/comments/1gtag3z/what_is_the_current_consensus_of_ai_submitted

[Rust-for-Linux] Rust for Linux Project https://rust-for-linux.com/ Retrieved on 2 JAN 2025

[Rust] Rust Team, Rust. Retrieved on 2 JAN 2025 from https://www.rust-lang.org/

[Sandström] Johannes Sandström, Can AI models solve the programming challenge Advent of Code? Evaluating state of the art large language models. Uppsala Universiteit, Retrieved on 22 DEC 24 from https://www.diva-portal.org/smash/get/diva2:1870236/FULLTEXT01.pdf

[Speed24] Richard Speed, "Torvalds weighs in on 'nasty' Rust vs C for Linux debate." 19 SEP 2024. The Register. Retrieved on 2 JAN 2025 from https://www.theregister.com/2024/09/19/torvalds_talks_rust_in_linux/

Growing System Engineers in the Age of ChatGPT © 2025 by William Bardon is licensed under CC BY-NC-SA 4.0  . To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/